- **Utiliser les commandes Artisan**
- **Gérer le cache Laravel**
- **Manipuler les collections Laravel**
- **Compiler et optimiser les assets (CSS, JS) avec Laravel Mix**
- **Comprendre et utiliser Contracts, Events, Helpers, Gestion fichiers**
- **Envoyer des emails et notifications**
- **Gérer la localisation, la queue et le task scheduling**

B.2.1 – Commandes Artisan

Artisan est l'outil en ligne de commande intégré à Laravel.

Commandes utiles :

php artisan list

Affiche toutes les commandes disponibles dans Laravel.
Utile pour explorer l'outil.

php artisan make:controller PropertyController

Crée un contrôleur PropertyController dans app/Http/Controllers.
Fichier généré : app/Http/Controllers/PropertyController.php

php artisan route:list

Affiche toutes les routes définies avec leurs méthodes HTTP et contrôleurs associés.
Permet de vérifier vos routes.

php artisan serve

Lance un serveur local sur http://127.0.0.1:8000.
Utile pour tester vos pages Laravel.

php artisan migrate

Exécute les migrations définies dans database/migrations pour créer les tables en base.

Astuce : php artisan help <commande> montre tous les paramètres possibles pour la commande.

B.2.3 – Cache Laravel

Laravel propose un système de **cache** pour stocker temporairement des données lourdes et améliorer la performance.

Exemple pratique

Fichier : routes/web.php

```
use Illuminate\Support\Facades\Cache; // Permet d'utiliser Cache

Route::get('/test-cache', function() {

    // Stocke la valeur 'valeur' dans le cache sous la clé 'key' pendant 3600 secondes (1 heure)

    Cache::put('key', 'valeur', 3600);


    // Récupère la valeur stockée

    $value = Cache::get('key', 'valeur_par_defaut'); // 'valeur_par_defaut' si clé absente


    // Supprime la clé du cache

    Cache::forget('key');


    return $value;

});
```

Explications ligne par ligne :

1. use Illuminate\Support\Facades\Cache; → Permet d'utiliser la façade Cache de Laravel.
2. Cache::put('key', 'valeur', 3600); → Stocke une donnée dans le cache pour 3600 secondes.
3. Cache::get('key', 'valeur_par_defaut'); → Récupère la donnée stockée, sinon retourne 'valeur_par_defaut'.
4. Cache::forget('key'); → Supprime la donnée du cache.

Astuce : Utiliser le cache pour les listes de biens, catégories ou menus statiques.

B.2.4 – Collections Laravel

Les **collections** sont des tableaux enrichis qui offrent de nombreuses méthodes.

Fichier : routes/web.php

```
use Illuminate\Support\Collection;
```

```
Route::get('/collections', function() {
    $properties = collect([
        ['title' => 'Appartement', 'price' => 120000],
        ['title' => 'Villa', 'price' => 350000],
        ['title' => 'Studio', 'price' => 80000]
    ]);

    // Filtrer les biens > 100000
    $expensive = $properties->where('price', '>', 100000);

    // Boucler et afficher le titre
    $expensive->each(function($item){
        echo $item['title'] . '<br>';
    });

    // Trier par prix décroissant
    $sorted = $properties->sortByDesc('price');
});
```

Explications :

- collect([...]) → Crée une collection Laravel à partir d'un tableau.
- where('price', '>', 100000) → Filtre les biens dont le prix > 100000.
- each(function($item){ ... }) → Boucle sur chaque élément de la collection.
- sortByDesc('price') → Trie la collection par prix décroissant.

Astuce : Les collections remplacent avantageusement foreach, array_map, array_filter.

B.2.5 – Compilation assets (Mix)

Laravel Mix compile CSS/JS avec Webpack.

```
npm install   # Installe toutes les dépendances
npm run dev   # Compilation pour le développement (non minifiée)
```

npm run prod # Compilation pour production (minifiée)

Fichiers concernés :

- resources/css/app.css
- resources/js/app.js
- Compilation → public/css/app.css et public/js/app.js

Astuce : Toujours compiler les assets avant production pour optimiser le temps de chargement.

B.2.6 – Contracts

Les **Contracts** sont des interfaces pour accéder aux services Laravel.

```
use Illuminate\Contracts\Cache\Repository as CacheContract;


class PropertyService {
    protected $cache;


    public function __construct(CacheContract $cache) {
        $this->cache = $cache;
    }
}
```

Explications :

- CacheContract → Interface de cache de Laravel.
- $this->cache → On code contre l'interface, donc changer de driver n'affecte pas le code.

B.2.7 – Events

Créer un event :

```
php artisan make:event PropertyAdded

php artisan make:listener SendPropertyNotification --event=PropertyAdded
```

Fichier Event : app/Events/PropertyAdded.php
Fichier Listener : app/Listeners/SendPropertyNotification.php

Dans un contrôleur :

```
event(new PropertyAdded($property));
```

Explications :

- event(...) déclenche l'event.
- Le listener attaché exécute la logique secondaire (ex: envoi email).

Schéma mental :

Utilisateur → Contrôleur → Event → Listener → Action secondaire

B.2.8 – Gestion fichiers (Storage)

```
use Illuminate\Support\Facades\Storage;


Route::get('/file-test', function() {
   Storage::put('file.txt', 'Contenu du fichier'); // Crée le fichier
   $content = Storage::get('file.txt');          // Lit le contenu
   Storage::delete('file.txt');                  // Supprime le fichier
   return $content;
});
```

Fichier : routes/web.php
Explications ligne par ligne :

- Storage::put() → Crée/écrit un fichier.
- Storage::get() → Lit le contenu du fichier.
- Storage::delete() → Supprime le fichier.

Astuce : Configurer .env pour le driver : local, s3, ftp.

B.2.9 – Helpers

- route('home') → Génère l'URL d'une route nommée
- asset('css/app.css') → URL vers un fichier public
- old('field') → Récupère la valeur précédente d'un formulaire
- now() → Renvoie la date et l'heure actuelles

B.2.10 – Localisation

```
// resources/lang/fr/messages.php
return [
    'welcome' => 'Bienvenue',
];

// Dans Blade
{{ __('messages.welcome') }}
```

Astuce : Traduction facile pour applications multilingues.

B.2.11 – Symfony Mailer

```
use Illuminate\Support\Facades\Mail;
use App\Mail\PropertyAddedMail;

Mail::to('user@example.com')->send(new PropertyAddedMail($property));
```

Explication :

- Mail::to()->send() → Envoie un email.
- PropertyAddedMail → Mailable généré via Artisan : php artisan make:mail PropertyAddedMail.

B.2.12 – Notifications

```
use App\Notifications\PropertyAdded;
use Illuminate\Support\Facades\Notification;

Notification::route('mail', 'user@example.com')->notify(new PropertyAdded($property));
```

- Notifications multi-canaux : email, SMS, Slack…

B.2.13 – Packages

```
composer require barryvdh/laravel-debugbar
```

Astuce : Vérifier compatibilité avec Laravel avant installation.

B.2.14 – Queue

```
php artisan queue:table
php artisan migrate
```

Dans un Job (app/Jobs/SendEmailJob.php) :

```
public function handle() {
    // Code à exécuter en arrière-plan
}
```

Astuce : Utiliser pour emails, exports lourds, notifications.

B.2.15 – Task Scheduling

```
// app/Console/Kernel.php
protected function schedule(Schedule $schedule) {
    $schedule->command('emails:send')->daily();
}
```

Astuce : Configurer CRON pour exécuter php artisan schedule:run toutes les minutes.

Partie C : Approfondir la programmation Laravel
C.1 Sécurité Laravel

- **Authentifier les utilisateurs**
- **Sécuriser les APIs (JWT, Passport, Sanctum)**
- **Hacher et chiffrer les données sensibles**
- **Autoriser les actions via Gates et Policies**
- **Réinitialiser les mots de passe**
- **Vérifier les emails**

C.1.1 – JWT / Passport / Sanctum

Exemple JWT (routes/api.php) :

```
Route::post('/login', function(Request $request) {
   $credentials = $request->only('email', 'password');

   if (!$token = auth()->attempt($credentials)) {
      return response()->json(['error'=>'Non autorisé'], 401);
   }

   return response()->json(['token'=>$token]);
});
```

Explications ligne par ligne :

- $request->only('email', 'password') → Récupère uniquement email et mot de passe du formulaire
- auth()->attempt($credentials) → Tente d'authentifier l'utilisateur
- response()->json([...], 401) → Renvoie erreur si non autorisé
- response()->json(['token'=>$token]) → Renvoie le token JWT si authentification réussie

C.1.2 – Authentification

Fichier : routes/web.php

```
Route::get('/dashboard', function() {
   return view('dashboard');
})->middleware('auth');
```

Explications :

- middleware('auth') → Protège la route, seul un utilisateur connecté peut y accéder
- /login et /register peuvent être générés via :

```
php artisan ui vue --auth
```

C.1.3 – Chiffrement

Fichier : n'importe quel contrôleur

```
use Illuminate\Support\Facades\Crypt;

$secret = Crypt::encryptString('Donnée sensible'); // Chiffre la donnée
$original = Crypt::decryptString($secret);         // Déchiffre la donnée
```

- APP_KEY dans .env → Clé utilisée pour chiffrer/déchiffrer

C.1.4 – Autorisation

Dans un contrôleur :

```
$this->authorize('update', $property);
```

- Vérifie si l'utilisateur courant peut update le modèle $property.
- Les **Policies** sont liées aux modèles et gèrent la logique d'autorisation.

C.1.5 – Hachage

```
use Illuminate\Support\Facades\Hash;

$hashed = Hash::make('motdepasse');   // Hachage du mot de passe
Hash::check('motdepasse', $hashed);   // Vérifie le mot de passe
```

- Toujours hacher avant de stocker en base

C.1.6 – Reset password

- /password/reset → Formulaire demande de réinitialisation
- /password/email → Envoi email
- /password/reset/{token} → Formulaire modification

Astuce : Vérifier configuration mail dans .env.

C.1.7 – Vérification email

Fichier : app/Models/User.php

```
use Illuminate\Contracts\Auth\MustVerifyEmail;


class User extends Authenticatable implements MustVerifyEmail {}
```

Route protégée :

```
Route::get('/dashboard', ...)->middleware(['auth', 'verified']);
```

- L'utilisateur doit confirmer son email pour accéder à la route